

ADA_P1_Acevedo_Contreras_Pena

Wiki del proyecto

Práctica 1 — Análisis y Diseño de Algoritmos
Módulos FB (Fuerza Bruta) y BT (Backtracking)

Nota: este PDF presenta una versión organizada y legible del contenido de la Wiki. Para consultar la versión original, actualizada y con la estructura interactiva del proyecto, ver la Wiki del repositorio.

Contenido

1. Informe Técnico
2. Uso de Inteligencia Artificial
3. Contribuciones individuales

Informe Técnico — Práctica 1 (Fuerza Bruta y Backtracking)

Estructura según la Sección 12 del enunciado (18 secciones).

Ver también: **Uso de Inteligencia Artificial y Contribuciones individuales.**

1. Portada

- **Título:** Práctica 1 — Fuerza Bruta y Backtracking
- **Curso:** Análisis y Diseño de Algoritmos, EAFIT
- **Integrantes (orden alfabético):** Acevedo Otálvaro, Juan José · Contreras, Jerónimo · Peña Rojas, Juan Esteban
- **Fecha de entrega:** *(completar con la fecha confirmada por el curso)*

2. Introducción

Esta práctica trabaja el mismo problema desde dos lados opuestos. En el módulo FB actuamos como un atacante que no conoce una contraseña y necesita medir cuánto cuesta encontrarla probando todo el espacio posible. En el módulo BT actuamos como quien diseña una política de contraseñas y necesita generar (o contar) las contraseñas válidas sin generar primero todo el espacio y filtrar al final.

La idea detrás de juntar los dos módulos es notar la diferencia entre "explorar todo sin ninguna estructura" (fuerza bruta) y "explorar descartando ramas apenas se sabe que no llevan a nada" (backtracking con poda). Este documento cubre el modelamiento de cada problema, el diseño de los algoritmos, su pseudocódigo, el análisis de complejidad, los casos de prueba usados, la experimentación real corrida sobre nuestras propias instancias, y la comparación entre enfoques que pide el enunciado.

3. Contexto del problema

Un sistema nunca guarda una contraseña en texto plano, guarda su hash (en este caso SHA-256). Si a un atacante se le filtra la base de hashes, la única forma garantizada de recuperar una contraseña dentro de un espacio conocido es probar cada candidato, calcular su hash y compararlo. Eso es fuerza bruta pura, y es exactamente lo que mide el módulo FB: cuánto cuesta ese ataque a medida que crece el alfabeto o la longitud de la contraseña.

El módulo BT ataca el problema desde el otro extremo: si una empresa exige que las contraseñas cumplan cierta política (mínimo de dígitos,

de mayúsculas, sin caracteres repetidos seguidos), ¿cómo se generan o cuentan las contraseñas válidas sin primero generar todo Σ^n y filtrar?

La respuesta es construir la contraseña carácter por carácter y abandonar una rama en cuanto se sabe que ya no puede cumplir la política — eso es backtracking con poda.

4. Fundamentación teórica

Fuerza bruta: un algoritmo de fuerza bruta genera y evalúa sistemáticamente todos los candidatos de un espacio de soluciones, sin usar ninguna propiedad del problema para descartar candidatos antes de evaluarlos. Es aplicable cuando el espacio es finito y enumerable y no existe una propiedad estructural que permita saltarse candidatos. Su costo crece con $|\Sigma|^n$ — cada carácter adicional multiplica el costo por el tamaño del alfabeto, no lo suma.

Backtracking: es también una técnica de búsqueda exhaustiva sobre un árbol de estados parciales, pero construye la solución de forma incremental y abandona una rama ("la poda") tan pronto una función de factibilidad determina que ningún candidato de esa rama puede llegar a ser válido. La diferencia con fuerza bruta no está en la garantía de completitud — ambas la tienen — sino en el orden de exploración y en usar información parcial para no explorar subárboles completos. Es importante no confundir esto con un cambio de complejidad: en el peor caso (una política casi vacía) backtracking explora un árbol de tamaño comparable al de la enumeración completa. Lo que la poda reduce es el número de nodos *efectivamente* visitados frente al tamaño teórico del árbol, y eso es justamente lo que este informe cuantifica en la Sección 14.

5. Modelamiento

FB. Espacio de búsqueda: para un alfabeto Σ ($A_1 = 26$ símbolos o $A_2 = 36$ símbolos) y longitud fija n , el espacio de candidatos tiene tamaño $|\Sigma|^n$. Un candidato es solución si y solo si $\text{SHA256}(\text{candidato}) = \text{hashObjetivo}$.

BT. Alfabeto de 67 caracteres (26 minúsculas, 26 mayúsculas, 10 dígitos, 5 símbolos !@#%\$). Estado parcial: el prefijo construido hasta el momento, junto con cuántos caracteres de cada tipo lleva (minúscula/mayúscula/dígito/símbolo). Estado inicial: cadena vacía. Estados terminales: cadenas de longitud n . La política del equipo

(derivada de la semilla 2124, ver Sección 10) es:

```
minLower = 2 + (2124 mod 3) = 2
minUpper = 1 + (2124 mod 2) = 1
minDigit = 1 + (2124 mod 3) = 1
minSymbol = 1
```

Como $2124 \bmod 3 = 0$ y $2124 \bmod 2 = 0$, la fórmula del enunciado da exactamente $\text{minLower}=2$, $\text{minUpper}=1$, $\text{minDigit}=1$, $\text{minSymbol}=1$ — la suma (5) no supera $n=8$, así que no hace falta reducir nada. También se prohíben dos caracteres iguales consecutivos, para todas las instancias.

6. Diseño algorítmico

FB — enumeración exhaustiva. Se trata cada candidato como un "odómetro" de base $|\Sigma|$: se arranca en $\Sigma[0]\Sigma[0] \dots \Sigma[0]$ y en cada paso se incrementa la posición más a la derecha, propagando acarreo cuando una posición se desborda — igual que contar en base $|\Sigma|$. Esto visita cada cadena de Σ^n exactamente una vez, sin omisiones ni repeticiones, porque hay una biyección entre los enteros de 0 a $|\Sigma|^n - 1$ (en base $|\Sigma|$) y las cadenas de Σ^n .

BT — construcción incremental con dos podas. La contraseña se construye carácter por carácter. En cada nodo se aplican dos podas antes de bajar a un hijo:

1. **Poda por consecutivos:** si la política prohíbe repetir el último carácter y el candidato a agregar lo repite, se descarta sin bajar a esa rama.
2. **Poda por factibilidad:** con los caracteres que faltan por poner (restante), se calcula cuánto le falta al prefijo para cumplir cada mínimo de la política. Si lo que falta supera lo que queda por poner, ningún candidato de esa rama puede ser válido — se poda. Los conteos de tipo se llevan de forma incremental (se actualiza solo el tipo del carácter agregado en cada paso, en vez de recomtar todo el prefijo en cada nodo), lo que baja el costo de la poda de $O(n)$ a $O(1)$ por nodo.

7. Pseudocódigo

7.1 FB — Búsqueda exhaustiva

```
función busquedaExhaustiva( $\Sigma$ , n, hashObjetivo):
    idx ← arreglo de n ceros
    candidatosEvaluados ← 0

    mientras verdadero:
        candidato ←  $\Sigma[\text{idx}[0]] \Sigma[\text{idx}[1]] \dots \Sigma[\text{idx}[n-1]]$ 
        candidatosEvaluados ← candidatosEvaluados + 1
```

```

si SHA256(candidato) = hashObjetivo:
    retornar (encontrada=verdadero, candidato, candidatosEvaluados)

pos ← n - 1
mientras pos ≥ 0:
    idx[pos] ← idx[pos] + 1
    si idx[pos] < |Σ|:
        romper el ciclo interno
    idx[pos] ← 0
    pos ← pos - 1

si pos < 0:
    retornar (encontrada=falso, candidatosEvaluados)

```

7.2 FB — Ataque por diccionario

```

función ataquePorDiccionario(archivo, hashObjetivo):
    candidatosEvaluados ← 0

    para cada línea candidato en archivo:
        candidatosEvaluados ← candidatosEvaluados + 1
        si SHA256(candidato) = hashObjetivo:
            retornar (encontrada=verdadero, candidato, candidatosEvaluados)

    retornar (encontrada=falso, candidatosEvaluados)

```

7.3 BT — Backtracking con poda

```

función backtrack(prefijo, n, política, lower, upper, digit, symbol):
    nodosVisitados ← nodosVisitados + 1

    si |prefijo| = n:
        si lower ≥ política.minLower y upper ≥ política.minUpper
        y digit ≥ política.minDigit y symbol ≥ política.minSymbol:
            soluciones ← soluciones + 1
        retornar

    restante ← n - |prefijo| - 1

    para cada carácter c en Σ:
        si política.noConsecutivos y prefijo no vacío y último(prefijo) = c:
            nodosPodados ← nodosPodados + 1
            continuar con el siguiente c

    (lower', upper', digit', symbol') ← (lower, upper, digit, symbol) + tipo(c)

    faltante ← max(0, política.minLower - lower')
        + max(0, política.minUpper - upper')
        + max(0, política.minDigit - digit')
        + max(0, política.minSymbol - symbol')

    si faltante > restante:
        nodosPodados ← nodosPodados + 1
        continuar con el siguiente c

    prefijo.agregar(c)
    backtrack(prefijo, n, política, lower', upper', digit', symbol')
    prefijo.quitarUltimo()

```

7.4 BT — Enumeración exhaustiva sin poda (para comparar)

```

función exhaustivo(prefijo, n, política):
    nodosVisitados ← nodosVisitados + 1

    si |prefijo| = n:
        si prefijo cumple política:
            soluciones ← soluciones + 1
        retornar

    para cada carácter c en Σ:
        prefijo.agregar(c)
        exhaustivo(prefijo, n, política)
        prefijo.quitarUltimo()

```

8. Implementación

- FB reutiliza una sola estructura SearchResult (encontrada,

contraseña, candidatos evaluados, tiempo) tanto para fuerza bruta como para diccionario — misma interfaz, sin duplicarla.

- BT lleva los conteos de tipo de carácter como parámetros de la recursión (`lower`, `upper`, `digit`, `symbol`), en vez de recalcularlos en cada nodo — es lo que baja la poda de $O(n)$ a $O(1)$ por nodo.

- Un solo `main.cpp` expone los comandos de ambos módulos (``hash`, `verify-ref`, `search`, `dict`, `gen-instances`, `bt`, `bt-compare``), como pide la Sección 11 del enunciado.

- SHA-256 vía `picosha2` (`third_party`, MIT) — no se implementó desde cero, como exige la Sección 10.

- La CLI captura las excepciones del programa (alfabeto inválido, archivo de diccionario inexistente) y las convierte en un mensaje de error controlado en vez de terminar abruptamente.

9. Análisis de complejidad

FB. Tiempo: $O(|\Sigma|^n)$ en el peor caso — se generan y evalúan las $|\Sigma|^n$ cadenas, y el costo de hashear un candidato es $O(1)$ para las longitudes acotadas de este trabajo. Espacio: $O(n)$, solo se guarda el candidato actual, nunca el espacio completo. Esto se confirma en la práctica: los candidatos evaluados coinciden exactamente con $|\Sigma|^n$ en las 7 configuraciones de peor caso corridas (Sección 11).

BT. El árbol completo (sin podar) tiene $\sum_{k=0}^n |\Sigma|^k$ nodos, con $|\Sigma|=67$. El peor caso de backtracking conserva esa misma cota exponencial — una política sin restricciones efectivas no poda nada, como se ve en la política `none` (Sección 14, reducción de apenas 2.94%). Lo que cambia con una política restrictiva es cuánto de ese árbol se visita de verdad: con la política `team` en $n=3$, la instancia es imposible ($2+1+1+1=5 > 3$) y se poda desde la raíz — reducción del 99.9997%. Espacio: $O(n)$ por la profundidad de la pila de recursión.

10. Casos de prueba

Instancia de referencia FB (común al curso): `abc12`, `A2`, $n=5$, hash `8d51feb34e3e69f6fa6dffc577e2c60490cf9a7fcd835f9f6af1505b71d74773`. Verificada con `verify-ref`: encontrada, 50,249 candidatos evaluados.

Semilla del equipo: apellidos Acevedo, Contreras, Peña (ya en orden alfabético), normalizados sin tildes/mayúsculas → `acevedocontreraspena` → suma de códigos ASCII mod 100000 → **2124**. Orden de alfabetos para

las 5 instancias de FB (A1,A2,A1,A2,A1) confirmado oficialmente por el profesor en el foro del curso.

n	Alfabeto	Contraseña	SHA-256
4	A1	slaz	d477fe9fdf5725d7ff0 91744b9662732735b32 165ba8978fa116f176b 5a25dfc
4	A2	y70p	dc319045ebdcd8bd3af 2e6459de2975233a05e 371cd5e7ec3f8b88a5a 43473a0
5	A1	ixuxs	c16a61c136f28aa6f95 c87747ab2f4a13ab710 6babf6f62db78732009 60c8bae
5	A2	bgdaf	d6809ac7b6f13c1d90a dd5ef7fb3f2b8599b11 2809c50ed19251d9d68 c14517f
6	A1	ixmbgr	70bebf0bcde96437c88 91522b46033087e12f7 bfad3421cd1f068b3ee 820b8c1

Política del equipo para BT: `minLower=2, minUpper=1, minDigit=1, minSymbol=1`, sin consecutivos iguales, derivada de la misma semilla

(Sección 5). La **instancia de referencia común de BT** que pide la

Sección 9.2 del enunciado (esa misma política, n=6, alfabeto

completo) coincide exactamente con la variante team, n=6 que ya se

corrió para este equipo (Sección 11) — no hizo falta correrla aparte.

Verificación automatizada: tests/test_fb.py, tests/test_bt.py.

Ambos compilan el binario y corren la CLI real, no simulan resultados.

11. Experimentación

11.1 FB — peor caso (7 configuraciones)

Alfabeto	n	Candidatos	Tiempo
A1	3	17,576	35.1 ms
A1	4	456,976	540.8 ms
A1	5	11,881,376	~14.6 s
A1	6	308,915,776	683.4 s (~11.4 min)
A2	3	46,656	57.2 ms

Alfabeto	n	Candidatos	Tiempo
A2	4	1,679,616	~2.0 s
A2	5	60,466,176	~74.3 s

Candidatos evaluados = exactamente $|\Sigma|$ en los 7 casos, sin excepción. (results/tiempos_fb.csv, reproducible con run_experiments.sh.)

11.2 FB — instancias propias del equipo

n	Alfabeto	Contraseña	Candidatos hasta encontrarla	Tiempo	% del espacio
4	A1	slaz	323,830	390.7 ms	70.9%
4	A2	y70p	1,163,464	1.41 s	69.3%
5	A1	ixuxs	4,074,193	5.02 s	34.3%
5	A2	bgdaf	1,963,446	2.41 s	3.2%
6	A1	ixmbgr	105,773,218	4.05 min	34.2%

11.3 BT — comparación con y sin poda (5 configuraciones + n=6)

Política	n	Nodos árbol completo	Nodos con poda	Reducción	Soluciones	Tiempo con poda	Tiempo sin poda
none	3	305,320	296,342	2.94%	291,852	4.85 ms	5.25 ms
relaxed	3	305,320	230,742	24.43%	226,252	6.72 ms	7.90 ms
team	3	305,320	1	99.9997%	0	0.0001 ms	6.41 ms
relaxed	4	20,456,441	16,934,574	17.22%	16,638,232	284.8 ms	475.9 ms
team	5	1,370,581,548	56,389,826	95.89%	51,916,800	2.55 s	37.00 s

team, n=6 (instancia de referencia común de BT) y none, n=6 se corrieron aparte por su costo: 345.76 s (~5.76 min) y 1,388.04 s (~23.13 min) respectivamente. soluciones_iguales=1 en todas las comparaciones — la poda nunca cambia el conteo de soluciones, solo el costo de encontrarlas. (Fuente completa: report/QA_FB_BT.md, Secciones 11–14.)

Validación independiente de dos casos (fuera del propio programa, por conteo combinatorio):

- team, n=5: $C(5,2) \cdot 3! \cdot 26^3 \cdot 10 \cdot 5$ – (casos con las 2 minúsculas)

repetidas seguidas) = $52,728,000 - 811,200 = 51,916,800$ ✓ coincide exacto con lo que produjo el programa.

- relaxed, $n=3$: $67^3 - 41^3 = 231,842$ cadenas con al menos una minúscula, menos 5,590 que violan la regla de no-consecutivos = 226,252 ✓ coincide exacto.

12. Resultados

Resultados detallados en la Sección 11. Archivos de respaldo:

results/tiempos_fb.csv, results/comparacion_fb_diccionario.csv, results/instancias_txt/ (FB); report/QA_FB_BT.md (BT — las corridas de BT quedaron documentadas ahí; pasarlas también a results/bt_resultados.csv y results/bt_comparacion.csv es una tarea pendiente del equipo, no afecta la validez de los números).

13. Análisis de resultados

FB. Los candidatos evaluados coinciden exactamente con $|\Sigma|^n$ en

los 7 casos de peor caso — confirma que la enumeración no omite ni repite candidatos. El factor de crecimiento medido al subir n en 1 va de $\times 15.4$ a $\times 46.7$ en A1 (teórico $\times 26$) y ronda $\times 36$ en A2 (teórico $\times 36$); la desviación en n pequeño se debe al costo fijo de arrancar el proceso, que pesa más cuando el tiempo total es de milisegundos.

BT. El caso team, $n=3$ (99.9997% de reducción) muestra el efecto más extremo de la poda: como la política exige al menos 5 caracteres, la imposibilidad se detecta en la raíz sin generar ni un solo hijo. El caso team, $n=5$ es el más representativo del uso real: 95.89% de reducción, ~ 14.5 veces más rápido con poda, y exactamente las mismas 51,916,800 soluciones que sin poda — la poda ahorra tiempo, no cambia el resultado. La política none (2.94% de reducción) confirma lo que dice la teoría: una política casi vacía no le da a la poda casi nada que descartar, y el árbol explorado se acerca al tamaño teórico completo.

14. Comparación algorítmica

FB vs. diccionario. El diccionario oficial (500 palabras) responde en menos de 1.5 ms siempre, sin importar el alfabeto o la longitud de la contraseña buscada, porque nunca genera candidatos: solo recorre una lista fija. Ninguna de las 5 instancias reales del equipo estaba en la lista (0/5 — se generan por LCG, no son palabras comunes), así que el ataque por diccionario habría fallado en los 5 casos aunque la fuerza

bruta sí los encontró todos. Como control, `access123` (que sí está en la lista, en la posición 484 de 500) se encuentra en menos de 1 ms incluso teniendo que recorrer casi todo el archivo — la ventaja del diccionario es la velocidad, su límite es que no es exhaustivo.

BT con poda vs. sin poda. Ver tabla completa en la Sección 11.3. En las 5 configuraciones comparadas, el número de soluciones es idéntico entre ambas versiones — verificación de correctitud exigida por la Sección 8.2 del enunciado. La reducción del espacio de búsqueda va desde casi nula (`none`, 2.94%) hasta casi total (`team`, $n=3$, 99.9997%), dependiendo de qué tan restrictiva sea la política.

15. Conclusiones

Fuerza bruta se comporta exactamente como predice la teoría: el costo crece como $|\Sigma|^n$, confirmado candidato por candidato en las 7 configuraciones medidas, y ese crecimiento deja de ser manejable en un computador personal alrededor de $n=6$ para A1 (11.4 minutos) — mucho antes para A2, por tener una base mayor. El ataque por diccionario es entre cientos y cientos de miles de veces más rápido, pero solo sirve si la contraseña real está en la lista; en nuestras 5 instancias propias nunca fue el caso, porque se generan por semilla, no por patrones comunes.

Backtracking con poda no cambia el orden de complejidad del problema — en el peor caso sigue siendo exponencial, como se ve en `none`, $n=6$ tardando 23 minutos — pero para políticas restrictivas como la del equipo, la poda reduce el trabajo real hasta en un 95-99%, sin cambiar el número de soluciones encontradas. Esa es la diferencia central que pedía demostrar la práctica: la poda no evita el peor caso teórico, pero sí evita pagarlo casi siempre en la práctica.

16. Referencias

- `picosha2` — biblioteca de SHA-256 de dominio público / MIT, autor okdshin. Usada en `src/third_party/picosha2.h`, sin modificaciones.

- Enunciado y rúbrica de la práctica:
`ADA_Practica1_FuerzaBruta_Backtracking.pdf` (raíz del repositorio).

- No se usó bibliografía externa adicional para el diseño de los algoritmos — ambos se diseñaron a partir de la definición del problema y la teoría vista en clase.

17. Uso de herramientas de Inteligencia Artificial

Ver **Uso de Inteligencia Artificial**

— declaración de los tres integrantes (Acevedo y Contreras con el registro completo desplegable ahí mismo; Peña con un uso más básico, solo depuración y preguntas conceptuales).

18. Contribución individual

Ver **Contribuciones individuales** —

el detalle completo de qué hizo cada integrante, con la actividad concreta y el respaldo verificable (commits, archivos, tests). Resumen de una línea por persona:

- **Acevedo Otálvaro, Juan José:** Módulo FB completo (implementación, experimentación, corrección del diccionario oficial, documentación).
- **Peña Rojas, Juan Esteban:** Módulo BT completo (implementación, poda, integración de comandos en la CLI compartida).
- **Contreras, Jerónimo:** Testing y QA de ambos módulos, documento de validación (`report/QA_FB_BT.md`), gráficas de BT.

Uso de Inteligencia Artificial

Declaración según lo que exige la Sección 14 del enunciado: herramienta, período y para qué se usó. Cada integrante usó una herramienta distinta, así que va por separado.

Integrante	Herramienta	Período
Acevedo Otálvaro, Juan José	Claude (Claude Code)	23 al 31 de agosto de 2026
Contreras, Jerónimo	ChatGPT (OpenAI)	agosto de 2026
Peña Rojas, Juan Esteban	ChatGPT (OpenAI)	agosto de 2026

Acevedo — Módulo FB

Se usó durante todo el desarrollo del módulo FB: para entender el enunciado, para limpiar y simplificar el código ya escrito, para revisar errores de compilación y comportamiento inesperado de la CLI, para armar el repositorio de GitHub y la wiki, y para organizar los resultados de la experimentación a partir de datos que el propio programa generó al ejecutarse. La IA no calculó ni inventó ningún número de la experimentación, solo los organizó y ayudó a interpretarlos.

Metodología del registro (cómo se armó, y qué distingue una sugerencia de la IA de una decisión aplicada al proyecto)

Este documento no corresponde a una transcripción literal de la conversación mantenida con la IA. Se construyó como un **registro cronológico y resumido del proceso de trabajo**, con el objetivo de dejar evidencia de cómo se utilizó la inteligencia artificial durante el desarrollo de la práctica.

El registro se elaboró a partir de las interacciones realizadas durante el desarrollo del proyecto y conserva los aspectos relevantes para entender el proceso: consultas realizadas, decisiones tomadas, explicaciones solicitadas, código propuesto o revisado, pruebas ejecutadas, resultados obtenidos, errores detectados y cambios realizados posteriormente.

¿Cómo se construyó cada entrada?

Cada entrada resume una etapa o interacción relevante y busca responder, de forma breve, a cuatro preguntas:

1. ¿Qué se estaba haciendo?

Se describe el problema, tarea o etapa del proyecto en la que se encontraba el equipo.

2. ¿Qué papel tuvo la IA?

Se indica si la IA fue utilizada para explicar conceptos, revisar una implementación, proponer una solución, analizar resultados, detectar posibles problemas, orientar una decisión o revisar el estado del proyecto.

3. ¿Qué se hizo finalmente?

Se registra la decisión o acción que tomó el equipo después de la interacción. Esto permite diferenciar una sugerencia de la IA de una decisión efectivamente aplicada al proyecto.

4. ¿Cuál fue el resultado?

Cuando corresponde, se incluyen las pruebas ejecutadas, resultados obtenidos, archivos modificados o conclusiones alcanzadas.

Criterio de registro

No todas las interacciones de la conversación se registran como entradas independientes. Se agruparon aquellas que pertenecen a una misma etapa del desarrollo para evitar convertir el documento en una copia innecesariamente

extensa de la conversación.

Por ejemplo, varias consultas relacionadas con la implementación de un mismo archivo pueden aparecer resumidas en una sola entrada cuando forman parte de una misma actividad.

Por otro lado, se conservaron como entradas independientes aquellas interacciones que produjeron una decisión, cambio, prueba o resultado relevante para el proyecto.

Diferencia entre aporte de la IA y trabajo del equipo

El registro busca distinguir entre:

- **Consulta o recomendación de la IA:** una explicación, propuesta o revisión realizada durante la conversación.
- **Decisión del equipo:** la decisión tomada por los integrantes después de analizar la recomendación.
- **Implementación:** código o cambios efectivamente incorporados al proyecto.
- **Validación:** pruebas ejecutadas por el equipo para comprobar que una implementación funcionaba.
- **Resultado experimental:** datos obtenidos al ejecutar los experimentos en el computador del equipo.

Por esta razón, que una entrada indique que la IA propuso o recomendó algo **no significa necesariamente que esa propuesta haya sido incorporada al proyecto**. Cuando fue posible, el registro especifica qué se terminó implementando y verificando.

Naturaleza de los resultados

Los resultados experimentales incluidos en este registro corresponden a ejecuciones realizadas sobre el proyecto y no a valores estimados por la IA.

Por ejemplo, cuando se registran tiempos de ejecución, cantidad de candidatos evaluados o contraseñas recuperadas, estos valores provienen de las ejecuciones realizadas por el equipo en su entorno de desarrollo.

La IA se utilizó posteriormente para ayudar a interpretar esos resultados, detectar inconsistencias y organizar su presentación.

Propósito del registro

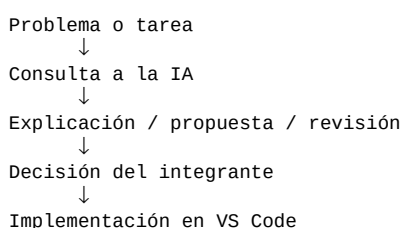
El propósito de este documento es proporcionar trazabilidad sobre el uso de IA durante el desarrollo de la práctica y permitir identificar:

- qué aspectos fueron consultados a la IA;
- qué tipo de asistencia se recibió;
- qué decisiones se tomaron durante el desarrollo;
- qué pruebas se realizaron;
- cómo se validaron las implementaciones;
- y cómo evolucionó el proyecto hasta su estado actual.

Por tanto, el documento debe entenderse como una **bitácora resumida del uso de IA durante el desarrollo**, y no como una conversación completa ni como un reemplazo de la documentación técnica del proyecto.

Flujo general utilizado

En términos generales, el uso de IA durante esta etapa siguió el siguiente flujo:



↓
Ejecución y validación local
↓
Resultado real
↓
Análisis y registro

Registro completo, entrada por entrada

Entrada 1 — Definición del registro de uso de IA

Se decidió mantener durante el desarrollo de la práctica una transcripción resumida del uso de IA, debido a que el profesor exige registrar cómo se utilizó la herramienta.

Se acordó que el registro no tendría que reproducir literalmente cada mensaje, sino documentar las consultas, decisiones, explicaciones, implementaciones, correcciones, pruebas y aportes relevantes realizados durante el desarrollo.

Entrada 2 — Comprensión inicial del documento

Se solicitó sintetizar y explicar el documento de la práctica de una manera más comprensible.

El equipo está conformado por tres integrantes:

- Un integrante encargado de BT.
- Un integrante encargado de FB.
- Un integrante encargado de testing.

Se decidió que el estudiante encargado de este registro desarrollaría el módulo **FB (Fuerza Bruta)**.

La IA explicó el contexto general de la práctica y enfocó la explicación hacia las responsabilidades específicas del módulo FB.

Entrada 3 — Inicio de la implementación

Después de revisar los conceptos teóricos vistos en clase, se decidió pasar directamente a la implementación.

Se estableció que inicialmente no existía código funcional del proyecto y que el desarrollo comenzaría desde cero.

Entrada 4 — Enumeración mediante índices

Se presentó el pseudocódigo de una búsqueda exhaustiva mediante un "odómetro" de base $|\Sigma|$.

El algoritmo utiliza:

- Un índice por cada posición de la contraseña.
- Inicialización de todos los índices en cero.
- Construcción del candidato actual.
- Cálculo de SHA-256.
- Comparación contra el hash objetivo.
- Incremento de la posición más a la derecha.
- Propagación del acarreo cuando una posición alcanza el tamaño del alfabeto.
- Terminación cuando se agota completamente Σ^n .

Se estableció esta estrategia como base de la implementación de FB.

Entrada 5 — Integración de SHA-256

Se indicó que la implementación de SHA-256 mediante `picosha2` ya había sido consultada y preparada previamente.

Se decidió utilizar un wrapper sencillo:

```
inline std::string sha256_hex(const std::string& input) {  
    return picosha2::hash256_hex_string(input);  
}
```

El objetivo fue mantener la integración sencilla y separar la función de hashing del algoritmo de búsqueda.

Entrada 6 — Implementación de alfabetos

Se implementó `fb_alphabet.hpp`.

Se definieron los dos alfabetos del proyecto:

```
A1 = abcdefghijklmnopqrstuvwxyz  
A2 = abcdefghijklmnopqrstuvwxyz0123456789
```

También se implementó una función para obtener el alfabeto a partir de su nombre (`a1` o `a2`), ignorando diferencias entre mayúsculas y minúsculas.

La implementación se mantuvo deliberadamente sencilla.

Entrada 7 — Implementación de Fuerza Bruta

Se implementó `fb_bruteforce.hpp`.

La implementación incluye:

- `SearchResult`.
- Indicador de éxito.
- Contraseña recuperada.
- Cantidad de candidatos evaluados.
- Tiempo de ejecución.
- Normalización del hash hexadecimal.
- Enumeración exhaustiva mediante índices.
- Propagación del acarreo.
- Medición del tiempo con `std::chrono`.

La estructura principal utilizada fue:

```
Σ■  
↓  
índices  
↓  
candidato  
↓  
SHA-256  
↓  
comparación  
↓  
incremento del odómetro
```


Se buscó mantener el código relativamente compacto sin sacrificar las métricas necesarias para la experimentación.

Entrada 8 — Implementación del CLI

Se implementó `main.cpp` como interfaz de línea de comandos.

Se incorporaron comandos para:

```
hash
verify-ref
search
dict
gen-instances
```

El programa permite:

- Calcular SHA-256.
- Verificar la implementación contra la instancia de referencia.
- Ejecutar búsqueda exhaustiva.
- Ejecutar ataque por diccionario.
- Generar las instancias propias del equipo.

También se implementó un parser sencillo para argumentos de tipo:

```
--clave valor
```

Entrada 9 — Implementación del ataque por diccionario

Se implementó `fb_dictionary.hpp`.

El algoritmo:

1. Abre `resources/diccionario.txt`.
2. Lee los candidatos línea por línea.
3. Ignora líneas vacías.
4. Calcula SHA-256 para cada candidato.
5. Compara con el hash objetivo.
6. Detiene la búsqueda al encontrar coincidencia.
7. Registra candidatos evaluados y tiempo.

Se reutilizó `SearchResult` para mantener una estructura común con Fuerza Bruta.

Entrada 10 — Generación de instancias

Se implementó `fb_seed.hpp`.

La implementación incluye:

- Normalización de apellidos.
- Eliminación de tildes.
- Conversión a minúsculas.
- Eliminación de caracteres no alfabéticos.

- Cálculo de la semilla del equipo.
- Generador LCG.
- Generación de contraseñas a partir del estado del LCG.

Se mantuvo el estado del LCG entre las diferentes contraseñas, de acuerdo con el procedimiento establecido para las instancias.

Entrada 11 — Primera validación de la implementación

Se ejecutó:

```
./ada_p1_fb verify-ref
```

Resultado:

```
Instancia de referencia: "abc12" sobre A2, n=5
Hash objetivo: 8d51feb34e3e69f6fa6dfffc577e2c60490cf9a7fcd835f9f6af1505b71d74773
Encontrada: SI
Contraseña recuperada: abc12
Candidatos evaluados: 50249
Tiempo (ms): 70.7228
[OK] Implementacion validada correctamente.
```

La instancia de referencia fue recuperada correctamente.

Esto permitió validar que la enumeración exhaustiva, el cálculo de SHA-256 y la comparación contra el hash estaban funcionando.

Entrada 12 — Prueba de caso negativo

Se decidió comprobar que Fuerza Bruta también terminara correctamente cuando el hash objetivo no perteneciera al espacio de búsqueda.

Se calculó el hash de:

```
abc1
```

y se buscó dentro de:

```
A13
```

Como abc1 tiene longitud 4, no puede pertenecer al espacio de contraseñas de longitud 3.

El hash obtenido fue:

```
dbfcfd0d87220f629339bd3adcf452d083fde3246625fb3a93e314f833e20d37
```

Se ejecutó:

```
./ada_p1_fb search --alphabet a1 --length 3 --hash dbfcfd0d87220f629339bd3adcf452d083fde3246625fb3a93e314f833e20d37
```

Resultado:

```
alfabeto=a1 |sigma|=26 n=3
encontrada=0
candidatos_evaluados=17576
tiempo_ms=21.3944
```

Se comprobó:

```
263 = 17.576
```

y el programa evaluó exactamente 17.576 candidatos.

Esto confirmó que el algoritmo agota completamente el espacio cuando no existe coincidencia.

Entrada 13 — Prueba positiva del diccionario

Se inspeccionó el archivo:

```
resources/diccionario.txt
```

La primera entrada era:

```
password
```

Se calculó su hash:

```
5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
```

Se ejecutó:

```
./ada_p1_fb dict --hash 5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
```

Resultado:

```
diccionario=resources/diccionario.txt
encontrada=1
password=password
candidatos_evaluados=1
tiempo_ms=0.126166
```

Como password era el primer candidato del archivo, se confirmó que el ataque por diccionario encuentra correctamente una coincidencia y detiene la búsqueda inmediatamente.

Nota posterior (29/08/2026): este resources/diccionario.txt resultó ser un archivo provisional, no el oficial de la práctica. El profesor mandó el archivo real por correo (mismo formato, 500 líneas, pero contenido distinto: password/qwerty/admin/letmein/etc.). Se reemplazó el recurso; esta entrada ya está actualizada con el resultado real contra el archivo oficial.

Entrada 14 — Validación de los dos algoritmos

Después de las pruebas anteriores se consideró validada funcionalmente la implementación de:

- Fuerza Bruta.
- Ataque por diccionario.

Se distinguieron los casos:

```
FB:
  caso positivo ✓
  caso negativo ✓

Diccionario:
  caso positivo ✓
```

Se aclaró que los tiempos de estas pruebas funcionales no debían utilizarse todavía como comparación de rendimiento, debido a que no representaban necesariamente las mismas instancias.

Entrada 15 — Apellidos y semilla del equipo

Los apellidos proporcionados para generar las instancias fueron:

```
Acevedo
Contreras
Peña
```

Ya estaban en orden alfabético.

Después de normalizarlos:

```
acevedo
contreras
pena
```

La cadena concatenada quedó:

```
acevedocontreraspena
```

La semilla calculada fue:

```
2124
```

Se verificó que la implementación de la semilla y del LCG coincidía con la especificación disponible.

Entrada 16 — Ambigüedad sobre el orden A1/A2

Se detectó que el documento indicaba que las instancias debían alternar entre A1 y A2, pero remitía a InteractivaVirtual para confirmar el orden.

Inicialmente no se encontró el orden específico.

Se decidió no asumirlo silenciosamente y consultar al profesor.

Entrada 17 — Confirmación oficial del orden de alfabetos

El profesor confirmó oficialmente que la secuencia debía ser:

```
Instancia 1 → A1, n=4
Instancia 2 → A2, n=4
Instancia 3 → A1, n=5
Instancia 4 → A2, n=5
Instancia 5 → A1, n=6
```

Esto confirmó que la implementación utilizada en `main.cpp` era correcta.

La razón adicional indicada por el profesor fue que A2 no se utiliza para longitud 6, por lo que la quinta instancia necesariamente debe utilizar A1.

Entrada 18 — Generación de las instancias propias

Se ejecutó:

```
./ada_p1_fb gen-instances --apellidos "Acevedo,Contreras,Peña"
```

Resultado:

```
Semilla del equipo = 2124
```

Las cinco instancias generadas fueron:

#	n	Alfabeto	Contraseña	SHA-256
-	-	-----	-----	-----
1	4	A1	slaz	d477fe9fdf5725d 7ff091744b96627 32735b32165ba89 78fa116f176b5a2 5dfc
2	4	A2	y70p	dc319045ebdcd8b d3af2e6459de297 5233a05e371cd5e 7ec3f8b88a5a434 73a0

#	n	Alfabeto	Contraseña	SHA-256
3	5	A1	ixuxs	c16a61c136f28aa6f95c87747ab2f4a13ab7106babf6f62db7873200960c8bae
4	5	A2	bgdaf	d6809ac7b6f13c1d90add5ef7fb3f2b8599b112809c50ed19251d9d68c14517f
5	6	A1	ixmbgr	70bebf0bcde96437c8891522b46033087e12f7bfad3421cd1f068b3ee820b8c1

Las contraseñas se conservaron para validación interna, mientras que los hashes se utilizaron como objetivos de búsqueda.

Entrada 19 — Validación de una instancia propia

Se decidió no ejecutar las cinco instancias propias individualmente porque el algoritmo ya había sido validado mediante:

- instancia positiva de referencia;
- caso negativo;
- generación reproducible de instancias;
- hash correcto de las instancias;
- prueba de una instancia propia.

Se ejecutó la primera instancia:

```
./ada_p1_fb search --alphabet a1 --length 4 --hash d477fe9fdf5725d7ff091744b9662732735b32165ba8978fa116f176b5a25dfc
```

Resultado:

```
alfabeto=a1 |sigma|=26 n=4
encontrada=1
password=slaz
candidatos_evaluados=323830
tiempo_ms=387.997
```

Se comprobó que:

```
26■ = 456.976
```

y que slaz fue encontrada antes de agotar el espacio.

Entrada 20 — Diseño de la experimentación

Una vez validada la implementación, se decidió pasar a la experimentación.

Se identificaron las siete configuraciones oficiales:

Alfabeto	n	Espacio
-----	-	-----
A1	3	17.576
A1	4	456.976
A1	5	11.881.376
A1	6	308.915.776
A2	3	46.656
A2	4	1.679.616
A2	5	60.466.176

Se decidió ejecutar las siete configuraciones para obtener una visión completa del crecimiento de la búsqueda exhaustiva.

También se distinguieron:

- experimentos de peor caso;
- instancias reales del equipo;
- comparación contra diccionario.

Entrada 21 — Resultados experimentales de FB

Se realizaron los experimentos correspondientes a las siete configuraciones.

Los resultados quedaron registrados en:

```
results/tiempos_fb.csv
```

También se generó la gráfica:

```
results/tiempo_vs_n_fb.png
```

Los resultados permitieron observar el crecimiento exponencial asociado al tamaño del espacio de búsqueda.

La configuración A1 con n=6 presentó un costo especialmente alto, llegando aproximadamente a 11,4 minutos en el peor caso.

Entrada 22 — Comparación FB vs. diccionario

Inicialmente se detectó que las mediciones de FB y diccionario estaban almacenadas en archivos separados.

Se aclaró que esto no significaba que las mediciones FB faltaran, sino que estaban registradas en:

```
results/tiempos_fb.csv
```

mientras que las mediciones del diccionario estaban en:

```
results/comparacion_fb_diccionario.csv
```

Se modificó `run_experiments.sh` para que la comparación ejecutara ambos métodos sobre el mismo hash y almacenara ambas mediciones en una misma fila.

El nuevo encabezado fue:

```
alfabeto,n,candidatos_fb,tiempo_fb_ms,encontrada_fb,candidatos_dict,tiempo_dict_ms,encontrada_dict
```

El caso de control (access123 desde que se reemplazó el diccionario, ver Entrada 13) quedó marcado como n/a para FB porque intentar Fuerza Bruta sobre una longitud 9 produciría un espacio de:

26■

lo cual no era una prueba razonable para este experimento.

La configuración A1 n=6 se mantuvo como medición independiente debido a su duración.

Entrada 23 — Congelación de resultados

Se ejecutó nuevamente el script de experimentación y se regeneraron los CSV con los resultados de la corrida correspondiente.

A1 n=6 no se volvió a ejecutar debido a su elevado tiempo de ejecución. Se utilizaron los resultados reales previamente obtenidos y se incorporaron a los archivos de resultados.

Se estableció una regla para el proyecto:

Los resultados experimentales quedan congelados y no se volverán a ejecutar salvo que se detecte un error real en el algoritmo, procedimiento experimental, datos o interpretación.

Entrada 24 — Revisión de reproducibilidad de las gráficas

Se detectó que:

results/tiempo_vs_n_fb.png

estaba desactualizada y contenía números correspondientes a una corrida anterior.

Se regeneró la gráfica utilizando los datos congelados de:

results/tiempos_fb.csv

Además, se creó:

plot_tiempos_fb.py

para que la gráfica pudiera reproducirse a partir del CSV.

También se incorporó:

plot_fb_vs_diccionario.py

para reproducir la gráfica de comparación entre Fuerza Bruta y diccionario.

Esto mejoró la reproducibilidad del proyecto y evitó depender de una gráfica generada manualmente.

Entrada 25 — Tablas y análisis experimental

Se construyeron cuatro tablas principales:

1. Crecimiento de Fuerza Bruta.
2. Instancias propias del equipo.
3. Comparación FB vs. diccionario.
4. Tasa de éxito.

También se analizaron:

- Diferencias entre A1 y A2.
- Efecto del incremento de n.

- Crecimiento teórico de $|\Sigma|$.
- Punto de impracticabilidad de Fuerza Bruta.
- Ventajas del ataque por diccionario.
- Limitaciones del ataque por diccionario.
- Comparación de tasas de éxito.
- Gráficas requeridas por el enunciado.

Se observó que A2 presenta un crecimiento más rápido porque:

```
|A1| = 26
|A2| = 36
```

por lo que incrementar la longitud multiplica el espacio de búsqueda por 26 en A1 y por 36 en A2.

También se identificó que la corrida larga de A1 $n=6$ puede presentar variaciones de rendimiento debido a condiciones del equipo, incluyendo posible thermal throttling.

Entrada 26 — Archivo adicional de resultados

Durante la revisión apareció un archivo que inicialmente no había sido creado directamente durante el flujo de trabajo:

```
results/fb_vs_diccionario.png
```

Se verificó que sus números coincidían con los CSV oficiales.

Se decidió conservarlo.

Posteriormente se añadió:

```
plot_fb_vs_diccionario.py
```

para hacer reproducible la gráfica correspondiente.

Entrada 27 — Congelación del repositorio

El desarrollo del módulo se hizo paso a paso, archivo por archivo, en el orden en que aparece en este registro (alfabeto, hash, fuerza bruta, CLI, diccionario, semilla, validación, experimentación, reproducibilidad de gráficas).

Ese avance no se fue registrando con commits a medida que se hacía — se trabajó de forma offline, sin llevar control de versiones durante el proceso. Al terminar, todos los cambios (implementación, `run_experiments.sh`, resultados, gráficas y sus scripts, README) se organizaron y se subieron al repositorio de una vez.

Se incluyeron:

- código fuente completo del módulo (`src/`);
- `run_experiments.sh`;
- `results/tiempos_fb.csv`;
- `results/comparacion_fb_diccionario.csv`;
- `plot_tiempos_fb.py`;
- `plot_fb_vs_diccionario.py`;
- `results/fb_vs_diccionario.png`;
- README actualizado.

Los cambios fueron finalmente **commiteados y pusheados** al repositorio.

Entrada 28 — Revisión del repositorio

Se revisó el repositorio:

jjacevedo/ADA_P1_Acevedo_Contreras_Pena

La estructura actual incluye:

```
src/
■■■ fb_alphabet.hpp
■■■ fb_bruteforce.hpp
■■■ fb_dictionary.hpp
■■■ fb_hash.hpp
■■■ fb_seed.hpp
■■■ main.cpp
■■■ third_party/
    ■■■ picosha2.h

resources/
■■■ diccionario.txt

results/
■■■ comparacion_fb_diccionario.csv
■■■ fb_vs_diccionario.png
■■■ tiempo_vs_n_fb.png
■■■ tiempos_fb.csv
■■■ instancias_txt/
    ■■■ a1_n3_peor_caso.txt
    ■■■ a1_n4_peor_caso.txt
    ■■■ a1_n4_instancia_equipo.txt
    ■■■ a1_n5_peor_caso.txt
    ■■■ a1_n5_instancia_equipo.txt
    ■■■ a2_n3_peor_caso.txt
    ■■■ a2_n4_peor_caso.txt
    ■■■ a2_n4_instancia_equipo.txt
    ■■■ a2_n5_peor_caso.txt
    ■■■ a2_n5_instancia_equipo.txt
    ■■■ comparacion_a1_n4.txt
    ■■■ comparacion_a1_n5.txt
    ■■■ comparacion_a2_n4.txt
    ■■■ comparacion_a2_n5.txt
    ■■■ comparacion_control_n9.txt

tests/
■■■ .gitkeep

report/
■■■ .gitkeep

plot_tiempos_fb.py
plot_fb_vs_diccionario.py
run_experiments.sh
README.md
```

results/instancias_txt/ se agregó después: la rúbrica exige un .txt por instancia en results/, además de los .csv y .png — run_experiments.sh ahora escribe uno por cada corrida, reusando la misma salida que imprime el CLI.

Se verificó que la implementación, resultados, scripts de reproducibilidad y documentación técnica están actualmente en el repositorio.

Entrada 29 — Reemplazo del diccionario por el archivo oficial del curso

El profesor envió un anuncio: el diccionario.txt (500 candidatos sintéticos) mencionado en el enunciado no estaba disponible, y un equipo lo reportó. El profesor adjuntó el archivo oficial y aclaró que ninguna de las 5 contraseñas propias del equipo debería aparecer ahí (se generan por semilla/LCG), y que eso no es un error — es justo lo que la Sección 8.1 pide discutir.

El `resources/diccionario.txt` que se venía usando hasta ese momento era un archivo provisional, con contenido distinto (incluía `acceso123` como caso de control). Se reemplazó por el archivo oficial (mismo formato: 500 líneas, palabras comunes en inglés — `password`, `123456`, `qwerty`, `letmein`, `admin`, `welcome`, `dragon`, `monkey`, `football`, `baseball`, `iloveyou`, `trustno`, `sunshine`, `master`, `shadow`, `superman`, `batman`, `princess`, `login`, `starwars`, `whatever`, `freedom`, `ninja`, `mustang`, `access` — cada una con 20 variantes).

Esto obligó a corregir en cadena:

- El caso de control pasó de `acceso123` a `access123` (sí está en el archivo oficial, en la posición 484 de 500).
- Se volvieron a correr `run_experiments.sh` y las pruebas manuales de diccionario contra las 5 instancias del equipo, confirmando que se mantiene 0/5 — la conclusión no cambió, solo los números específicos del diccionario.
- Se actualizaron `README.md`, la nota guardada para el informe, y esta misma entrada del registro (Entrada 13 tenía un ejemplo con la primera palabra del archivo viejo).

Se dejó constancia de que las 5 instancias del equipo (candidatos evaluados, tiempos de fuerza bruta) no cambian con este reemplazo — dependen del alfabeto/semilla, no del diccionario. Solo cambiaron las columnas relacionadas con el diccionario en `comparacion_fb_diccionario.csv`.

Contreras — Testing y QA

Se usó como apoyo para entender conceptos de fuerza bruta, backtracking, poda y funciones hash; para interpretar errores y comportamientos inesperados de la CLI (incluyendo diferencias de consola en Windows); para organizar la estructura de los tests automatizados; y para ayudar a interpretar resultados que ya habían sido obtenidos ejecutando el programa real. El código del proyecto y los resultados experimentales fueron ejecutados y verificados por el equipo, no generados por la IA.

[Reporte completo](#)

1. Herramienta utilizada

Herramienta: ChatGPT (OpenAI)

Periodo de uso: agosto de 2026.

La herramienta se utilizó como apoyo durante el desarrollo, pruebas y documentación de la práctica. El código del proyecto y los resultados experimentales fueron ejecutados y verificados en el entorno del equipo.

2. Usos realizados

2.1 Consulta y explicación de conceptos

Se utilizó ChatGPT para solicitar explicaciones sobre conceptos relacionados con:

- Fuerza Bruta;
- Backtracking;
- poda;
- espacios de búsqueda;
- complejidad exponencial;
- funciones hash SHA-256;
- comparación entre exploración exhaustiva y exploración con poda.

Estas consultas tuvieron como objetivo mejorar la comprensión de conceptos utilizados en la práctica.

2.2 Análisis de errores y comportamiento inesperado

Se utilizó ChatGPT para interpretar errores observados durante la ejecución de comandos del proyecto, por ejemplo:

- excepciones producidas por entradas inválidas;
- comportamiento de la CLI;
- diferencias de codificación de caracteres en la consola de Windows;
- interpretación de resultados de los comandos de BT;
- identificación de posibles problemas en los scripts de pruebas.

Las modificaciones realizadas al repositorio fueron efectuadas y verificadas por los integrantes.

2.3 Diseño y revisión de casos de prueba

ChatGPT fue utilizado como apoyo para organizar casos de prueba y proponer una estructura de automatización externa para tests/.

Los tests fueron ejecutados por el equipo sobre el código del repositorio y sus resultados fueron comprobados directamente.

2.4 Análisis de resultados

Se utilizó ChatGPT para ayudar a interpretar resultados que previamente habían sido obtenidos mediante ejecuciones reales del programa.

Por ejemplo, se analizaron:

- número de candidatos;
- nodos visitados;
- nodos podados;
- soluciones encontradas;
- tiempos de ejecución;
- porcentaje de reducción de nodos;
- crecimiento observado al aumentar n.

No se utilizaron resultados generados por la IA como sustituto de las ejecuciones experimentales.

2.5 Apoyo en la documentación

ChatGPT fue utilizado como apoyo para:

- organizar la documentación QA;
- estructurar el README;
- organizar tablas de resultados;
- redactar explicaciones técnicas basadas en resultados reales;
- estructurar el reporte de contribuciones;
- preparar la sección de uso de IA.

3. Uso de la IA en los resultados experimentales

Los resultados reportados en la práctica fueron obtenidos ejecutando el programa en el equipo del proyecto.

Ejemplos:

```
BT team, n=5
soluciones = 51,916,800
```

```
BT relaxed, n=3
soluciones = 226,252
```

```
BT team, n=5
```

reducción = 95.8857 %

BT none, n=6

tiempo ≈ 23.13 minutos

Estos valores fueron observados en ejecuciones reales y posteriormente analizados/documentados.

4. Alcance y límites

La IA no fue utilizada para:

- generar y entregar sin comprensión el algoritmo de Fuerza Bruta;
- generar y entregar sin comprensión el algoritmo de Backtracking;
- fabricar resultados experimentales;
- alterar resultados obtenidos;
- inventar instancias o tiempos para sustituir las ejecuciones;
- presentar como propias conclusiones que los integrantes no puedan explicar.

El equipo conserva la responsabilidad sobre el código, las ejecuciones, los resultados y las conclusiones presentadas.

5. Declaración

El uso de IA se realizó como herramienta de apoyo para comprensión, diagnóstico, organización de pruebas y redacción técnica. Los integrantes revisaron y verificaron el contenido utilizado en el proyecto.

Peña — Módulo BT

Uso básico: consultas puntuales de depuración (errores de compilación y comportamiento inesperado del código) y preguntas conceptuales sobre backtracking y poda. No se usó para diseñar el algoritmo ni para generar resultados — la implementación de `src/bt_backtracking.hpp` y la experimentación de BT son trabajo propio, ejecutado y verificado por él.

Nota sobre usos permitidos

El enunciado (Sección 14) permite consultar conceptos teóricos, pedir explicaciones de errores de compilación o comportamiento inesperado del código propio, y recibir retroalimentación sobre redacción. Prohíbe pedirle a una IA que genere el código fuente de los algoritmos, que calcule instancias o resultados que la IA no ejecutó, o presentar como propio un análisis que el equipo no pueda defender oralmente.

Vale aclarar con honestidad: en el caso de Acevedo, el uso fue más allá de consultas puntuales — la IA participó activamente en la limpieza del código y la documentación del módulo FB, no solo en resolver dudas puntuales, aún así esta no generó código por sí sola, solo apoyó el paso a paso hecho por el estudiante. Esa es justamente la razón por la que el registro existe con tanto detalle: para que cualquiera pueda revisar exactamente qué se hizo y por qué, y para que el equipo pueda defenderlo en la sustentación con la misma claridad con la que está documentado acá, sin el incumplimiento de los usos permitidos de la IA.

Referencia a la versión original

Este documento es una representación organizada para facilitar su lectura e impresión. El contenido corresponde al material de la Wiki del proyecto.

Para consultar la versión original, más organizada y actualizada, ver la Wiki del repositorio de GitHub.